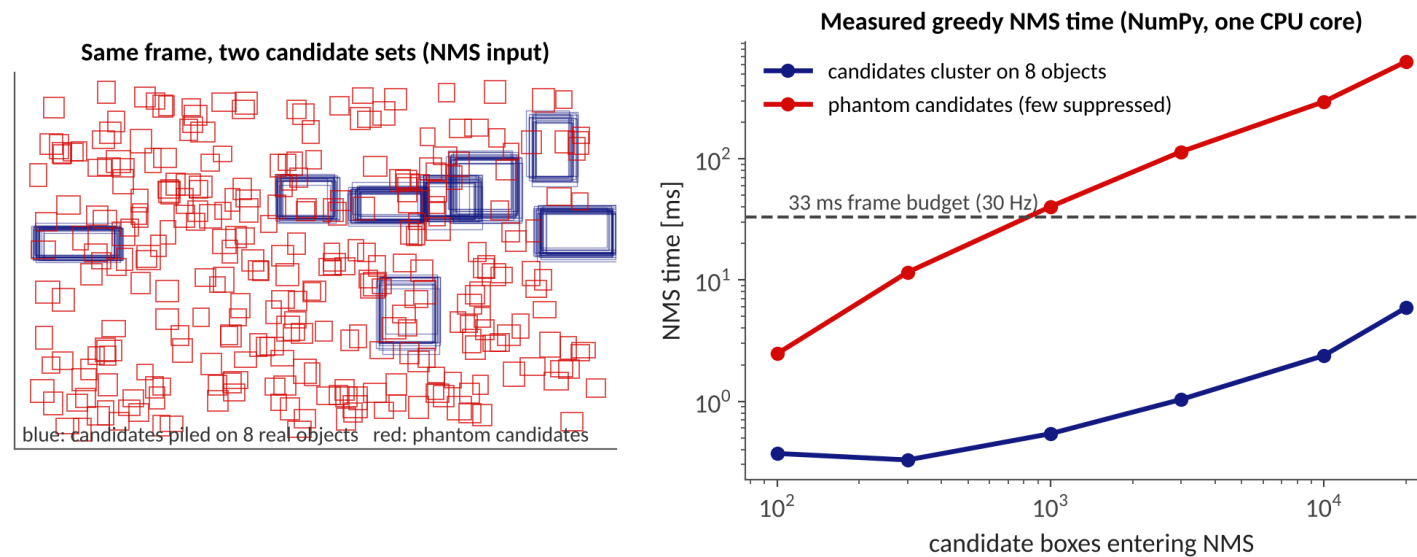


## L4.3 Latency / Availability Attacks for AD



Zonghua Gu | Fall 2026

# Outline

---

- Why availability matters for AD
- Latency attacks on perception
- Latency attacks on VLMs and LLMs
- Defenses: work budgets
- From model-level cost to system failure
- Open problems and discussion

# Integrity vs. Availability

|                          | Integrity attack (L3.2, L3.3)                           | Availability / latency attack (today)               |
|--------------------------|---------------------------------------------------------|-----------------------------------------------------|
| Attacker goal            | Wrong output: misclassification, hidden object, phantom | Late output, or none: more work, energy, or delay   |
| Typical metric           | Attack success rate, mAP drop                           | Amplification factor, deadline-miss rate, tardiness |
| Output when attacked     | Different from clean                                    | <b>Often identical to clean</b> , just delayed      |
| Why it is hard to notice | Accuracy monitors fire                                  | Task-accuracy monitors stay silent                  |
| Victim resource          | The decision                                            | Time, energy, GPU/CPU share, queue capacity         |

# AD Perception Lives Under a Deadline

**50-100 ms**

typical per-frame  
perception budget

**200 ms**

extra latency

**+6 m**

travelled before braking  
at 30 m/s (108 km/h)

## Chatbot

A slow answer is an annoyance; the user  
waits or retries

No physical consequence

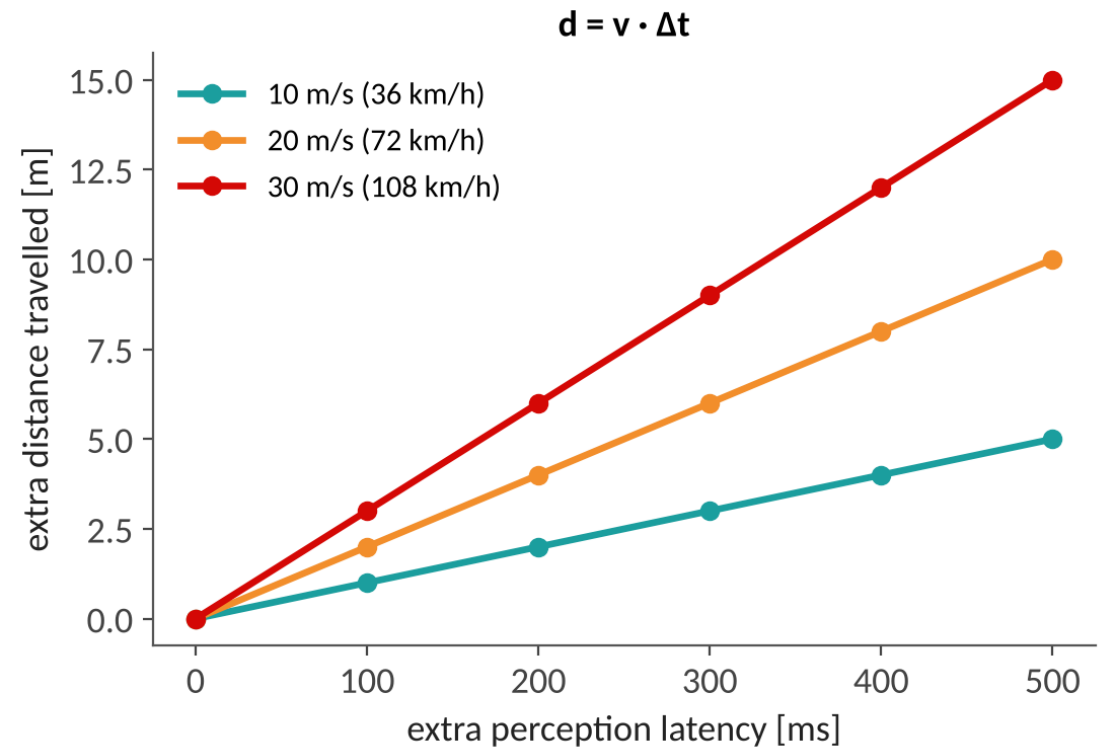
## Autonomous vehicle

A missed deadline means a stale or missing  
decision

Attack surface is physical: roadside patch,  
projected light, billboard

# Delay Becomes Distance

- Every extra millisecond of perception latency is **distance travelled blind**:  $d = v \cdot \Delta t$
- At 30 m/s, 100 ms costs **3 m**, 300 ms costs **9 m**
- The planner may also act on a stale world model: objects have moved since the frame was captured
- Questions for this lecture: how large a delay can an attacker cause, and is that delay big enough to matter?

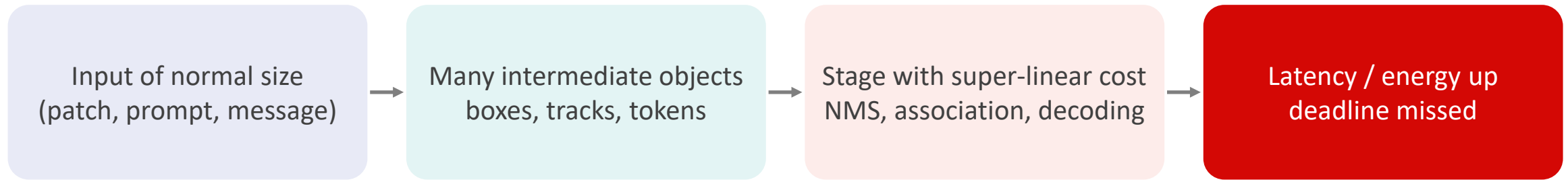


# What Is a Latency Attack?

---

- Goal: **inflate inference-time work, energy, or response time** rather than (only) cause a wrong answer
- Also called energy-latency, **sponge**, slowdown, or resource-consumption DoS attacks
- Outcome: deadline misses, throughput collapse, resource exhaustion, sometimes while **preserving the nominal prediction**
- Unifying mechanism: **intermediate-work amplification**: a small input drives many more boxes, tracks, tokens, or steps through a stage whose cost grows super-linearly
- Caution: 'worst case' is reserved for proven maxima; most papers find **high-cost inputs**, not true worst cases

# Intermediate-Work Amplification



- **NMS:** cost grows with the number of candidate boxes admitted
- **Tracker association:** cost grows with tracks  $\times$  detections, and false tracks persist across frames
- **Autoregressive decoder:** cost grows with the number of generated tokens (reasoning tokens included)
- Different from request flooding or oversized inputs: **the input looks benign in size**

# Metrics: Report More Than a Ratio

| Quantity                                 | Definition                                                           | Why it matters for AD                                                             |
|------------------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Amplification factor</b>              | $AF = M(x_{adv}) / \text{median } M(x_0)$ over matched benign inputs | Same model, batch size, hardware, and concurrency; avoids cherry-picked baselines |
| <b>Deadline-miss increase</b>            | $p_{attack} - p_{clean}$ (absolute, preferred)                       | Real-time systems have near-zero clean miss rate, so the ratio is undefined       |
| <b>Slack and tardiness</b>               | $S = d - f$ ; $\tau = \max(0, f - d)$                                | How late, not only whether late                                                   |
| <b>Consecutive misses, recovery time</b> | Longest run of missed frames; time to recover after the attack stops | Stateful pipelines (trackers) stay degraded after the input is gone               |

# Delivery Channels

| Delivery channel               | What the attacker controls             | Perception / AD examples                                                |
|--------------------------------|----------------------------------------|-------------------------------------------------------------------------|
| Digital input                  | Image, point cloud tensor              | Phantom Sponges, Overload, SlowTrack, SlowLiDAR, TrackFlood             |
| Physical sensor delivery       | Printed patch, lens sticker, projector | SlowPerception, DetStorm (Groundswell proposes it; evaluated digitally) |
| V2V message                    | Shared sensor data or features         | CP-FREEZER (cooperative perception)                                     |
| Prompt, API, retrieved content | Text or image sent to a served model   | Verbose Images, ReasoningBomb, OverThink                                |
| Training data or backdoor      | Training samples                       | Sponge backdoor on detectors; reflection backdoor on a driving VLM      |
| Weights or hardware            | Parameter bits                         | Rowhammer bit flips on NMS behavior (20.4×, 31 flips)                   |

# Bottleneck vs. Delivery Channel

## Bottleneck: what makes it expensive

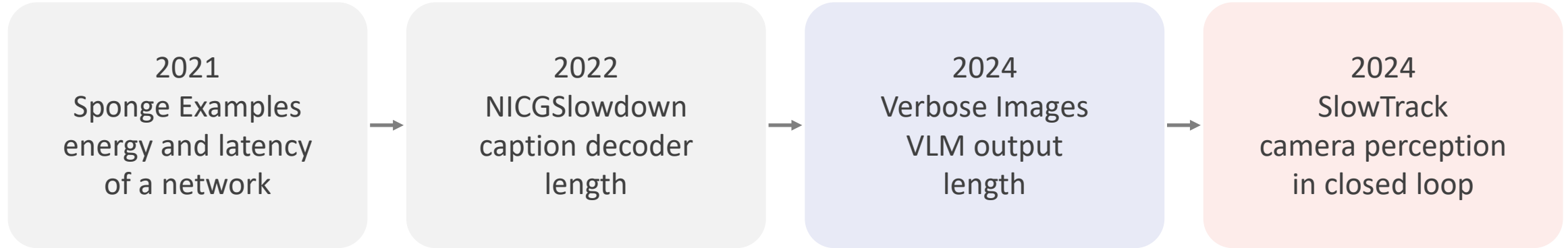
NMS candidates  
Tracker state  
LiDAR preprocessing, cooperative fusion  
Activation sparsity, early exit  
Output / reasoning length

## Delivery channel: how it is triggered

Digital input, physical patch or projector  
V2V message, prompt or API  
Retrieved content  
Training poison or backdoor  
Weight or hardware tampering

- The two axes are **orthogonal**: any bottleneck can be reached through several channels
- The channel decides **persistence, detectability, and cost**; the bottleneck decides **which control bounds the work**
- This lecture is organized by bottleneck

# How the Field Got Here



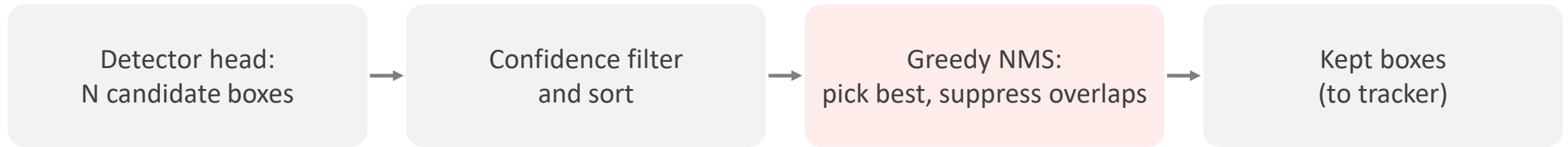
- Early work: a **single model**, measured in FLOPs or energy
- Then: **pipeline stages** around the model (NMS, tracker) and their real-time consequences
- Now: **physical delivery, cooperative perception, and VLM/LLM-based driving** (this lecture)

# Outline

---

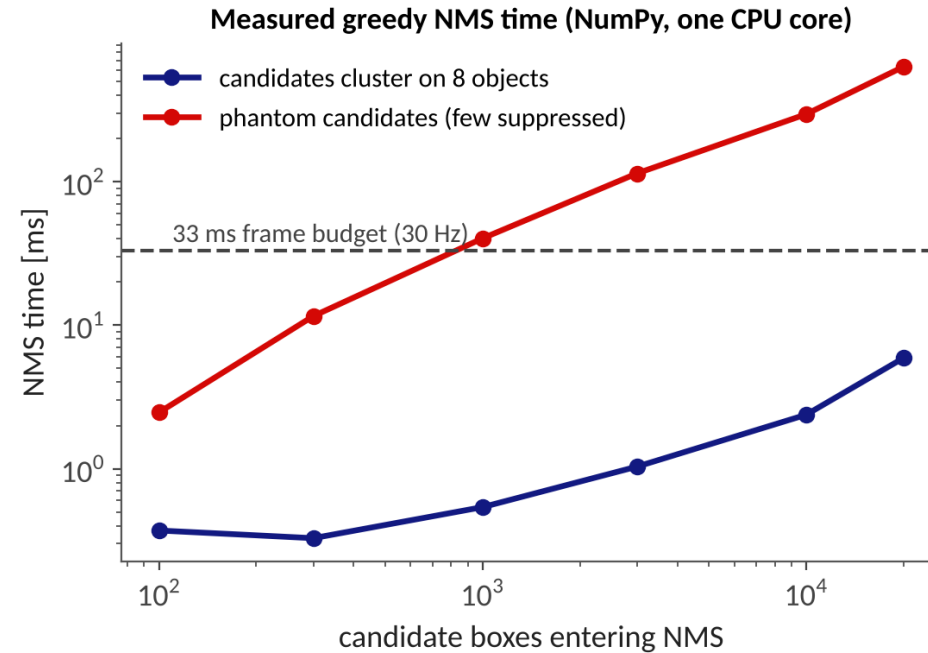
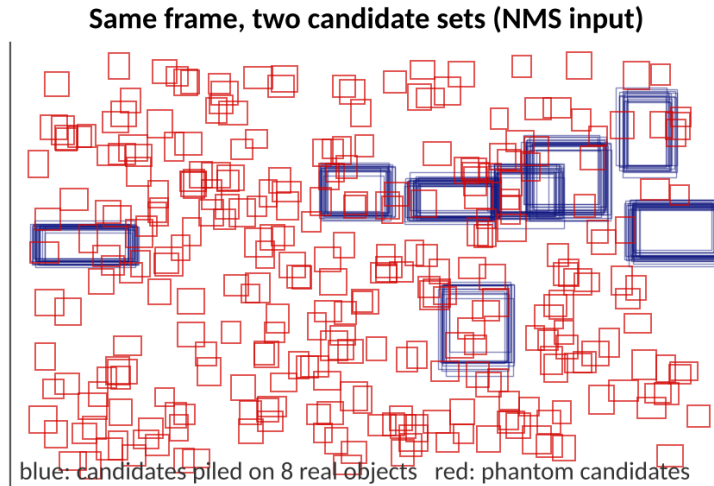
- Why availability matters for AD
- **Latency attacks on perception**
- Latency attacks on VLMs and LLMs
- Defenses: work budgets
- From model-level cost to system failure
- Open problems and discussion

# NMS: The Quadratic Bottleneck



- Benign scene: boxes **cluster** on a few objects, so each pick removes many candidates
- Attack: the perturbation adds many **small, spread-out phantom boxes** that overlap nothing, so almost none are suppressed
- Worst case grows like  $n^2$  comparisons in the number of admitted candidates
- Phantom Sponges keeps the original detections: **accuracy does not betray the attack**
- What matters for cost is how many candidates reach NMS, not how many leave it

# Measured: Phantom Candidates Blow the Budget



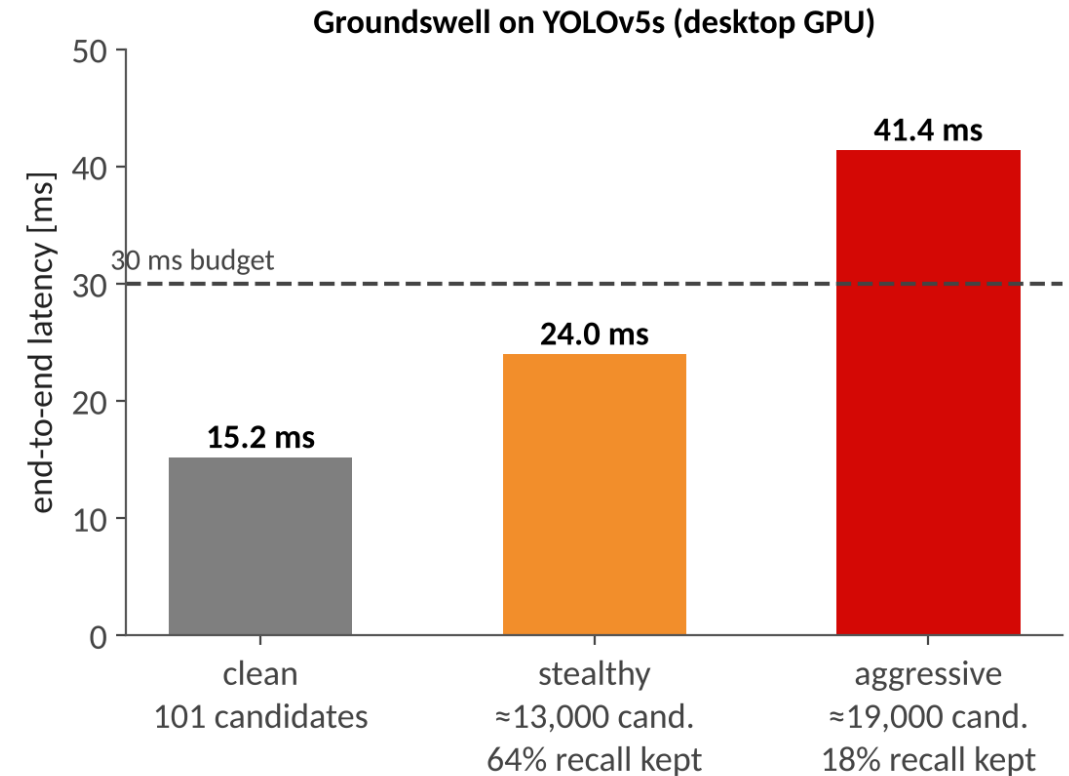
- Our NumPy greedy-NMS micro-benchmark, one CPU core: at **1,000 candidates** the phantom set costs **40 ms** vs. **0.5 ms** when boxes cluster on 8 objects, already above a 33 ms frame budget
- At 20,000 candidates: **633 ms** (phantom) vs. **5.9 ms** (clustered): a ~100× gap at the same n
- Caveat: real detectors cap the candidates they pass on, and GPU kernels differ (see the EVADE re-evaluation)

# Attacks on NMS

| Attack                                | Idea                                                                          | Reported result (native metric)                                   |
|---------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Phantom Sponges (WACV 2023)</b>    | Universal perturbation adds many phantom boxes, original detections preserved | NMS overloaded; detections preserved                              |
| <b>Overload (CVPR 2024)</b>           | Latency maximization with spatial attention, NMS-agnostic                     | Inference time up to 10× on edge devices                          |
| <b>Beyond PhantomSponges (2024)</b>   | Box-area loss lowers IoU, exacerbates NMS load                                | Up to +550 % NMS time (YOLOv5-small)                              |
| <b>Groundswell (VehicleSec 2026)</b>  | Regional Perturbation Balance places phantoms with minimal interference       | Candidates 101 → ~19k; 15.2 → 41.4 ms                             |
| <b>Steal Now, Attack Later (2024)</b> | Black-box ghost objects from reusable components                              | Works on common detectors and a commercial API, < \$1 per example |

# Groundswell: Stealth vs. Strength

- Same attack, two operating points on YOLOv5s (desktop GPU)
- **Stealthy**: ~13,000 candidates, **64 %** of original detections kept, 24.0 ms: still under a 30 ms budget
- **Aggressive**: ~19,000 candidates, only **18 %** recall kept, 41.4 ms: over budget, but the detector is largely blinded
- A **188×** rise in candidates gives only **2.7×** end-to-end latency: NMS is a small part of the pipeline (Part 5)



# Physical Delivery: What Is Actually Shown

| Attack                              | Physical channel                                             | Evidence level         | Reported                                      |
|-------------------------------------|--------------------------------------------------------------|------------------------|-----------------------------------------------|
| <b>DetStorm (IEEE S&amp;P 2025)</b> | Projected / placed objects, multiple NMS variants            | Target hardware        | +506 % detected objects; delays up to 8.1 s   |
| <b>SlowPerception (2024)</b>        | Projector universal perturbation loading NMS and the tracker | Closed-loop simulation | Avg. 2.5 s latency; 97 % collision rate       |
| <b>Groundswell (2026)</b>           | Lens sticker or projection (proposed)                        | Digital, GPU-timed     | Patches not optimized for physical deployment |

Physical delivery is shown in pieces (projector + hardware timing, or projector + simulator). The survey notes that no entry denotes a full on-road deployment.

# Reality Check: EVADE (NeurIPS 2025)

**114**

adversarial datasets

**15 / 7**

YOLO models / hardware platforms

**25k vs 8k**

pre-NMS proposal cap  
YOLOv5 vs. YOLOv8

## Findings

Attacks do not transfer across YOLO versions or sizes  
Universal patches fail across datasets  
Three defenses removed the effect on post-NMS counts  
Analytically, GPU NMS cannot reach 100-500 ms budgets

## Limits

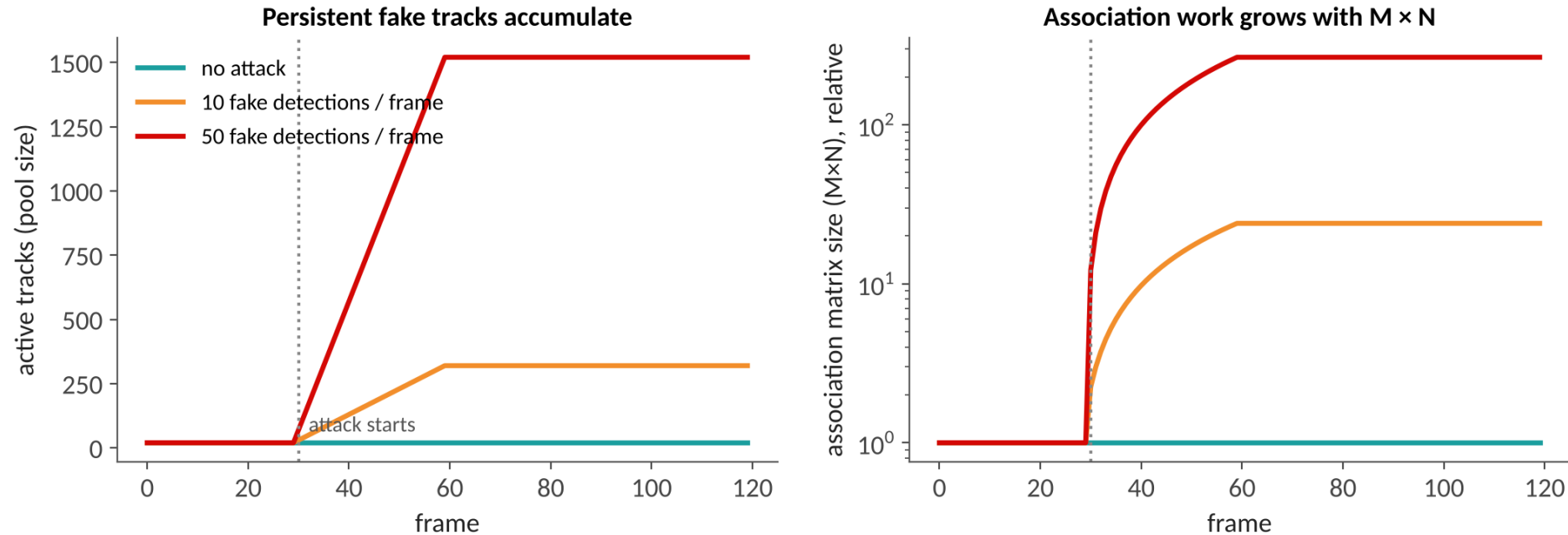
One detector family, one metric (post-NMS box count)  
DetStorm excluded (no public code)  
Not shown: that such attacks are impossible  
Amplification belongs to an attack, model, configuration triple

# Tracking: A Second, Stateful Bottleneck

---

- Tracker (SORT / ByteTrack style): predict each track, **associate** detections to tracks, update, spawn, retire (L3.3)
- With  $M$  tracks and  $N$  detections, building the association matrix costs  **$O(M \cdot N)$** ; assignment can be higher (e.g.,  $O(\max(M, N)^3)$  for Hungarian)
- Key difference from NMS: the tracker has **memory**. Unmatched false tracks survive for  $R$  frames, so work **accumulates** across frames
- Pool can grow toward  **$R \times |D|_{\max}$**  even if per-frame detections are capped
- Insight: detector-only attacks often fail because tracking absorbs noise (> 98 % detection success may be needed, Jia et al. 2020); latency attacks go **through** the tracker instead

# Why Per-Frame Caps Do Not Stop Accumulation



- Toy model (original): attack starts at frame 30; fake tracks expire  $R = 30$  frames after birth; 20 real objects
- Pool saturates at about  $R \times$  (fake detections per frame) and association work grows **hundreds of times**
- A cap on detections per frame bounds one dimension of  $M \times N$ , not the persistent pool

# SlowTrack: Latency Through the Tracker

- Camera-based perception, **two-stage** attack with three new loss designs
- Exploits the confirmation delay for new tracks and the grace period before a lost track is deleted, so the pool grows without bound
- Avg. **453.8×** slowdown of the tracking stage and **28.4×** for the whole camera pipeline (2.9× prior attacks)
- On the authors' Apollo + LGSVL setup: crash rate **~95 %** vs. ~30 % for baselines



# SlowTrack on a Jetson: Pool and Latency

---

**20.5 → 1,678**

mean tracker pool  
(tracks)

**3.3 → 140 ms**

tracking-stage  
latency

**57.2 → 1.1**

MOTA  
(tracking quality)

- Jetson AGX Orin, YOLOX-S + ByteTrack: **our group's measurement** of SlowTrack's effect
- Not quality-preserving: the tracker is both **slow and wrong**
- Still effective under a **500-detection cap**: per-frame caps do not bound the persistent pool

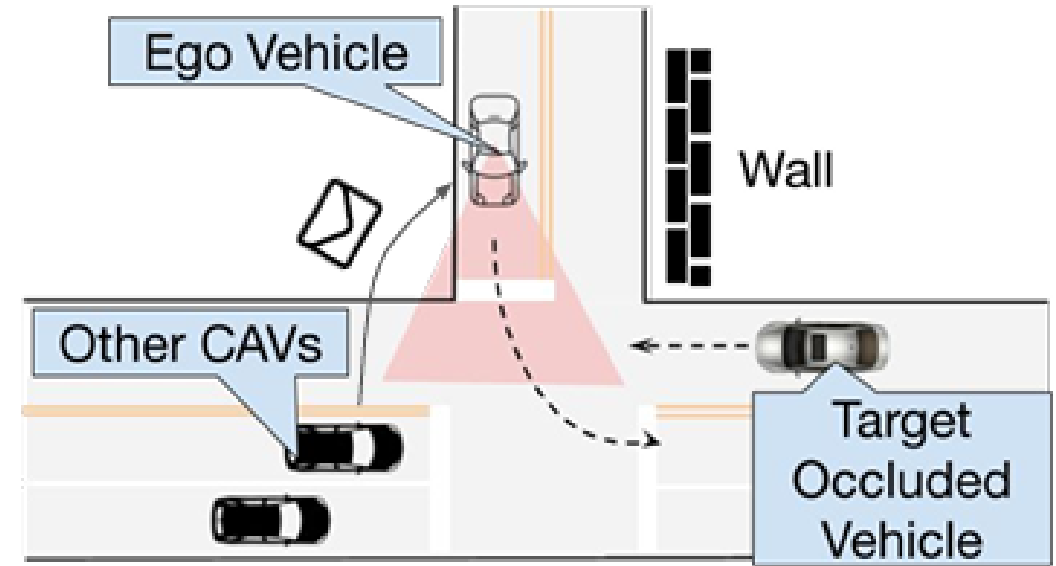
# TrackFlood: Remove NMS, Keep the Problem

---

- Newer detectors drop NMS: **YOLOv10 / YOLO26** (one-to-one heads) and **RT-DETR** (query-based)
- TrackFlood (our group) recovers each detector's per-slot confidences and floods the **tracker** with spatially spread phantom detections
- Jetson AGX Orin, TensorRT FP16: **detector latency stays 0.99-1.01×** (fixed number of slots); tracker latency rises
- 8/255 universal perturbation: end-to-end only **1.09-1.47×**, **no missed 33 ms deadlines**; 32/255 stress test: **13.54×**, every deadline missed
- Lesson: the bottleneck **relocates**; NMS-centric defenses have nothing to act on

# LiDAR and Cooperative Perception

- **SlowLiDAR** (CVPR 2023): first systematic availability attack on LiDAR detection; differentiable proxies for non-differentiable preprocessing; slows six pipelines
- **CP-FREEZER** (AAAI 2026): perturbed **V2V** messages maximize fusion delay: **> 90× latency, > 3 s per frame**, 100 % success on a real vehicle testbed
- Trust boundary moves from the sensor to the V2X link (cf. L3.3 data fabrication)



# Perception Latency Attacks at a Glance

| Method         | Bottleneck         | Delivery         | Evaluated       | Key result (native)         |
|----------------|--------------------|------------------|-----------------|-----------------------------|
| Overload       | NMS                | Digital          | Target hardware | up to 10× inference time    |
| Groundswell    | NMS                | Digital          | GPU-timed       | 101 → 19k candidates; 2.7×  |
| SlowTrack      | Tracker            | Digital          | Closed-loop sim | 28.4× pipeline; ~95 % crash |
| SlowPerception | NMS + tracker      | Projector        | Closed-loop sim | 2.5 s; 97 % collision       |
| DetStorm       | NMS                | Physical objects | Target hardware | up to 8.1 s delay           |
| TrackFlood     | Tracker (NMS-free) | Digital          | Target hardware | 1.09-1.47× at 8/255         |
| SlowLiDAR      | LiDAR runtime      | Digital          | Digital model   | latency up on 6 pipelines   |
| CP-FREEZER     | Cooperative fusion | V2V message      | Vehicle testbed | > 90×; > 3 s per frame      |

Native metrics and each paper's own baseline: rows are not comparable. Note how few reach closed-loop or testbed evaluation.

# Outline

---

- Why availability matters for AD
- Latency attacks on perception
- **Latency attacks on VLMs and LLMs**
- Defenses: work budgets
- From model-level cost to system failure
- Open problems and discussion

# Why VLM / LLM Components Change the Picture

---

- Foundation models are entering driving: **VLM-based perception, scene description, and planning** (LiU talk)
- Their cost is **output-dependent**: every generated token is one more decoder step, and reasoning models add thousands of reasoning tokens
- Compared with a CNN detector, the attacker gains a **new, wide lever**: output length is controllable by the input
- Compared with a chatbot, the victim has a **hard deadline**: there is no 'wait for the answer'
- Same attack mechanism as in perception, different bottleneck: more tokens instead of more boxes

# The Output-Length Lever

| Attack                     | Target           | Mechanism                                           | Reported (native metric)                 |
|----------------------------|------------------|-----------------------------------------------------|------------------------------------------|
| NMTSloth (FSE 2022)        | Neural MT        | Delay EOS with one character or token change        | Latency +85 % to +3,153 % (CPU)          |
| NICGSlowDown (2022)        | Image captioning | Imperceptible image perturbation lengthens captions | Latency up to +483.86 %                  |
| Engorgio (ICLR 2025)       | Open LLMs        | Optimized prompt suppresses EOS                     | 2-13× longer outputs, > 90 % of limit    |
| LoopLLM (AAAI 2026)        | LLMs             | Repetition loops until the length cap               | > 90 % of cap vs. ~20 % for EOS-delay    |
| Verbose Images (ICLR 2024) | VLMs             | Adversarial image induces long, repetitive text     | 7.87× / 8.56× length; latency ~2× to 24× |

For attacks that run to the limit, the factor is set by (output limit) / (benign length): a **deployment parameter**, not an attack property.

# Reasoning Models: Infinite Thinking

| Attack                   | Delivery            | Reported (native metric)                             | Note                                                         |
|--------------------------|---------------------|------------------------------------------------------|--------------------------------------------------------------|
| ReasoningBomb (CCS 2026) | Prompt              | Avg. 18,759 completion tokens; 6-7× more than benign | The 286.7× input/output ratio is not an amplification factor |
| ThinkTrap (NDSS 2026)    | Black-box API       | 50-100× more tokens                                  | Infinite-thinking loops                                      |
| OverThink (2025)         | Retrieved content   | Up to 46× (context-agnostic), 7.8× (context-aware)   | Answers stay correct                                         |
| RA-ICA (2025)            | Poisoned RAG corpus | Up to 13.1× tokens, > 90 % success                   | Answer integrity intact                                      |

Because several of these attacks keep the answer correct and bypass detectors, the survey argues for budget enforcement **by construction**, not detection.

# Reaching the Driving Stack

---

- **Natural-reflection backdoor on a driving VLM** (Liu et al., 2025): a reflection pattern in the camera image makes the planner generate abnormally long responses: a measured rise in output tokens and latency
- Delivery is a **training-time backdoor**; the trigger is something that occurs on real glass
- **Adjacent, not latency**: FreezeVLA (76.2 % freeze rate) and Semantic-DoS (up to 98.3 % hard-stop) **deny action** without showing amplified computation
- Distinguish: **late** (work amplification) vs. **absent** (denial of action) vs. **wrong** (integrity)

# Outline

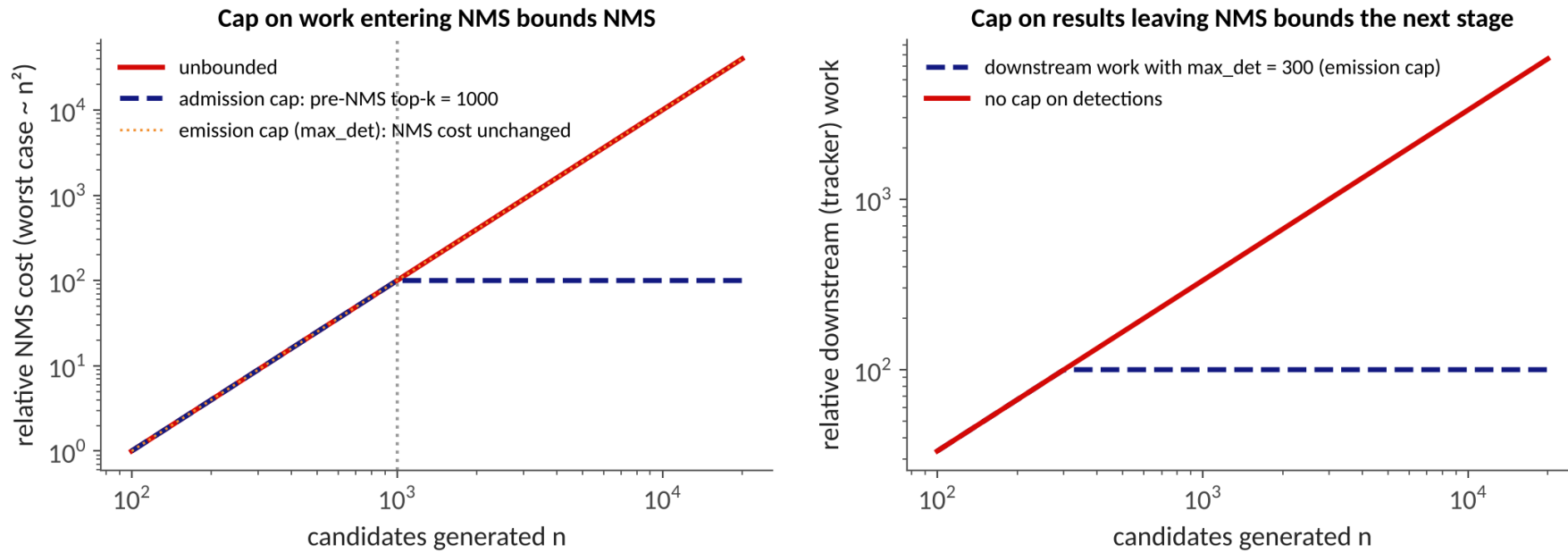
---

- Why availability matters for AD
- Latency attacks on perception
- Latency attacks on VLMs and LLMs
- **Defenses: work budgets**
- From model-level cost to system failure
- Open problems and discussion

# Defense Families by Control Mechanism

| Control mechanism                 | What it bounds                           | Examples                                      | Limits                               |
|-----------------------------------|------------------------------------------|-----------------------------------------------|--------------------------------------|
| Budget enforcement                | Work entering or leaving a stage; tokens | Pre-NMS top-k, token cap, runtime termination | Caps truncate hard legitimate inputs |
| Bounded / constant-time execution | Per-stage work or timing variation       | Admission caps, constant-time NMS             | Hardware effects still vary time     |
| Input transformation              | Perturbation reaching the model          | Adaptive resize, JPEG, purification           | Weak against adaptive attackers      |
| Robust / adaptive training        | Model behavior                           | Adversarial training for slowdown             | Efficiency-robustness trade-off      |
| Runtime monitoring                | Anomalous internal counts                | Track-count anomaly scores                    | Needs thresholds; adaptive evasion   |
| System / serving control          | Scheduling, queues, isolation            | Admission control, rate limits                | Not specific to perception           |

# Admission Cap vs. Emission Cap



- **Admission cap** (before the expensive stage): pre-NMS top-k, tracks admitted to association, token budget in the decoding loop: bounds that stage's **own** work
- **Emission cap** (after it): e.g., `max\_det` in Ultralytics: bounds the **next** stage's input, but NMS already did its full work
- Autoregressive decoding is where the two coincide: a max-token limit caps both (for the decode phase)

# Case: max\_det Looks Like a Fix, But...

**152-300 → 6-8**

post-NMS boxes under attack,  
max\_det 300 → 10

**5**

benign count under  
the same cap

## What it shows

A single emission cap bounds the detection count handed to the tracker and planner  
Independent of the attack's loss or perturbation model

## What it does not show

That NMS work or latency fell (the metric is censored at the cap)  
Safety in crowded scenes: 10 detections is too few  
A bound on the persistent tracker pool

# TrackShield: Admission Caps in the Tracker

**140 → 6.3 ms**

tracking-stage latency  
under SlowTrack

**100 % → 0.7 %**

deadline-miss rate  
at a 50 ms threshold

**~1 ms**

per-frame overhead

- Inside the ByteTrack update: hard **admission guard** before the cost matrix is built; caps at the **99th percentile** of benign counts
- Plus an EWMA / MAD anomaly monitor that limits track births and prunes likely phantom tracks
- Jetson AGX Orin, YOLOX-S + ByteTrack (our group's measurement)
- Limits: upstream re-identification is unprotected; accuracy is not restored; tested against transferred, not adaptive, attacks

# Input Transformation: Cheap, but Check the Adversary

---

- Adaptive input resizing: **-90.18 %** NMS time on an attacked YOLOv5-large (Beyond PhantomSponges)
- EVADE: an off-the-shelf diffusion purifier (PDM-Pure) returned the post-NMS box count to the **benign 5**; plain **JPEG** was nearly as effective (but less consistent across models)
- Both were tested only against **non-adaptive** attackers; the metric is an output count, not NMS time
- Where an adaptive attacker was tested (BPDA / EOT against an edge-cloud gate), **all twelve** training-free preprocessing baselines failed: attacked offload  $\geq 99.8 \%$
- Takeaway: input transformation raises the bar for existing attacks, not for adaptive ones

# Budgets for Reasoning: Hard vs. Soft

| Control (reasoning length)                            | Strength | Guarantee                                     |
|-------------------------------------------------------|----------|-----------------------------------------------|
| Runtime termination, token accounting                 | Hard     | Enforces a configured upper bound             |
| Parameter-level (CoT-Valve, learned length direction) | Medium   | Reduces observed length; no certified maximum |
| Prompt-level (TALE, Concise CoT)                      | Soft     | Encourages brevity; can be removed or ignored |

A cap has no bypass rate and needs no attack examples; its cost is paid by benign inputs that legitimately exceed it, and the operator chooses that cost. Use **difficulty-adaptive** budgets for genuinely hard problems.

# Timing Is Also a Side Channel

---

- Data-dependent runtime leaks information: NMS time **reveals internal inference state** and can sharpen black-box, decision-based attacks (Biton et al., AISec 2023)
- Opposite goal from latency attacks (confidentiality, not availability), **same observable**: variable runtime
- Motivates **constant-time NMS**: removes both the latency lever and the timing leak within the implementation boundary
- But: even with fixed nominal operations, wall-clock time varies through memory access, caching, contention, and thermal state

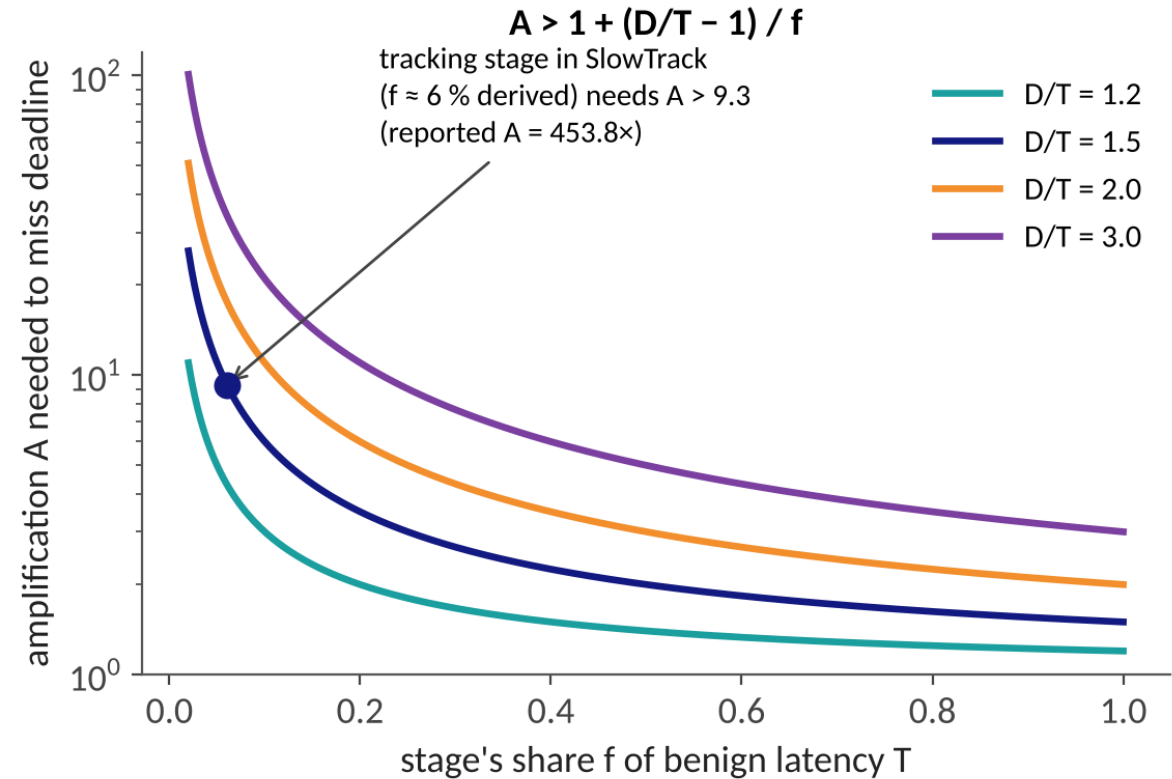
# Outline

---

- Why availability matters for AD
- Latency attacks on perception
- Latency attacks on VLMs and LLMs
- Defenses: work budgets
- **From model-level cost to system failure**
- Open problems and discussion

# When Does Amplification Miss a Deadline?

- Pipeline with benign latency  $T$ ; one stage has benign share  $f$  and is amplified by  $A$
- Sequential stages:  $A_{e2e} = 1 + f(A - 1)$
- Deadline  $D$  is missed only if  $A > 1 + (D/T - 1) / f$
- Neither  $f$  nor  $D/T$  is a property of the model: they belong to the **system**



# Worked Example: Big Stage Factor, Smaller Pipeline

**453.8x → 28.4x**

SlowTrack: tracking stage  
vs. whole pipeline

**≈188x → 2.7x**

Groundswell: NMS candidates  
vs. end-to-end latency

**1.0-1.3x**

tracker slowdown behind  
NMS-free detectors (ByteTrack)

- The unamplified stages **dilute** the effect: stage-level factors overstate pipeline-level risk
- The attacker can also **choose the path**: AESOP: 2,407× FLOPs targeting the execution path vs. 117× for the single most vulnerable model
- Conversely, a modest factor can be fatal near a **queueing knee** (GateDrain: 2.16× demand, 2.26-12.8× benign p99)

# Six Conditions Between Model and System

| Condition                     | What decides it                                      | Evidence                                            |
|-------------------------------|------------------------------------------------------|-----------------------------------------------------|
| <b>C1 Critical-path share</b> | Benign share $f$ of the amplified stage; concurrency | 453.8× stage vs. 28.4× pipeline                     |
| <b>C2 Headroom vs. slack</b>  | Ceiling on the amplified count; D/T                  | TrackFlood: no miss at 8/255, all missed at 32/255  |
| <b>C3 Clipping</b>            | Existing caps and timeouts                           | AESOP: throughput collapse or 96.7 % data discarded |
| <b>C4 Accumulation</b>        | State and queues across frames                       | Tracker pool 20.5 → 1,677.7                         |
| <b>C5 Sharing</b>             | Co-tenancy, shared queues                            | 2.16× demand → 2.26-12.8× benign p99                |
| <b>C6 Consequence mapping</b> | Fallback policy; closed-loop dynamics                | ~95 % crash vs. a safe stop in a different design   |

# Exercise: Is This Attack Dangerous?

## Given

Pipeline  $T = 30$  ms, deadline  $D = 45$  ms

Tracker share  $f = 5\%$

Attack amplifies the tracker by  $A = 50$

## Compute

Threshold:  $1 + (D/T - 1) / f = ?$

$A_{e2e}$  and the new latency = ?

Deadline missed?

## Then change one thing

An admission cap limits  $A$  to 5

What is the new  $A_{e2e}$ ?

## Discuss

The fallback is a safe stop instead of continuing: is the outcome a collision or lost availability?

Which of C1-C6 did you just vary?

# Real-Time Systems View: AI Has No WCET

---

- Classical real-time design guarantees timing through **worst-case execution time (WCET)** analysis and schedulability tests
- AI-based perception has **no WCET**: execution time depends on the input, and attacks search for high-cost inputs, not proven maxima
- Admission caps turn input-dependent work into a **bounded** quantity; a runtime guard bounds elapsed time but converts overrun into missing or partial results
- Design options: bound work, bound time, add a **fallback** (safe stop, degraded mode), isolate resources across tasks

# A Better Evaluation Checklist

| Report with every amplification factor                                      | Why                                                             |
|-----------------------------------------------------------------------------|-----------------------------------------------------------------|
| Benign latency $T$ , deadline $D$ , stage share $f$                         | Lets a reader compute whether Eq. (4) is satisfied              |
| Ceiling on the amplified count; existing caps and timeouts                  | Bounds what the attack can reach (C2, C3)                       |
| Hardware, resolution, confidence threshold, pre-NMS cap, NMS implementation | Amplification is a property of attack, model, and configuration |
| Absolute deadline-miss increase, slack, consecutive misses, recovery time   | Real-time outcome, not a ratio                                  |
| Fallback policy and terminal outcome                                        | Crash vs. safe stop (C6)                                        |
| Attacker cost: one-time and per-delivery                                    | Cost-impact pair; delivery realism                              |

# Outline

---

- Why availability matters for AD
- Latency attacks on perception
- Latency attacks on VLMs and LLMs
- Defenses: work budgets
- From model-level cost to system failure
- **Open problems and discussion**

# Open Problems

---

- **Physical realizability** end to end: most physical results are partial (projector + simulator, or hardware timing); no full on-road deployment
- **NMS-free and other new detectors**: TrackFlood shows the bottleneck relocates; within-detector surfaces (feature necks, deformable attention, token pruning) are **unvalidated conjectures**
- **Bound the worst case** of VLM / VLA perception; **schedulability analysis** for AI-based pipelines; **anytime** models with deadline-aware early exit (LiU talk)
- **Adaptive** attacker evaluation for every defense; standardized, system-level benchmarks with downstream outcomes

# Discussion Questions

---

- Why can an attack that preserves every detection still be a **safety** attack? Give a timeline.
- A vendor claims '100× slowdown found in our detector'. Which five numbers do you ask for?
- Where would you put a cap in an Apollo-like stack: before NMS, before association, or at the controller input? What does each protect, and what does it cost?
- Safe stop vs. keep driving on stale data: which fallback turns a latency attack into an **availability** failure rather than a **collision**?
- How would an **adaptive** attacker respond to the cap you chose?

# Reading List

---

- Survey: Gu, Gao, Saremi, Chakraborty, Deep Learning Latency Attacks and Defenses (arXiv:2609.36732); companion site: [github.com/guzonghua/awesome-latency-attacks](https://github.com/guzonghua/awesome-latency-attacks)
- Shumailov et al., **Sponge Examples** (EuroS&P 2021)
- Ma et al., **SlowTrack** (AAAI 2024) and **SlowPerception** (arXiv 2024)
- Monteuuis et al., **Latency NMS Attacks: Real Life or Fantasy?** (NeurIPS 2025)
- Wang et al., **CP-FREEZER** (AAAI 2026); Muller et al., **DetStorm** (IEEE S&P 2025)
- Gu et al., **TrackFlood** and **TrackShield** (2026): our group's work on trackers

# References

---

- Gu, Gao, Saremi, Chakraborty. Deep Learning Latency Attacks and Defenses. arXiv:2609.36732, 2026.
- Shumailov et al. Sponge Examples: Energy-Latency Attacks on Neural Networks. EuroS&P 2021.
- Shapira et al. Phantom Sponges. WACV 2023; Chen et al. Overload. CVPR 2024; Schoof et al. Beyond PhantomSponges. WiseML 2024.
- Xia, Gao, Shi. Stealthy Latency Attacks Against Camera-Based Object Detection. VehicleSec 2026.
- Ma et al. SlowTrack. AAI 2024; SlowPerception. arXiv:2406.05800; Muller et al. DetStorm. IEEE S&P 2025.
- Monteuuis, Chen, Petit. Latency NMS Attacks: Is It Real Life or Is It Just Fantasy? NeurIPS 2025.
- Liu et al. SlowLiDAR. CVPR 2023; Wang et al. CP-FREEZER. AAI 2026; Jia et al. Fooling Detection Alone Is Not Enough. ICLR 2020.
- Gu et al. TrackFlood arXiv:2609.33948; TrackShield, HPCC 2026; GateDrain arXiv:2609.33992.
- Gao et al. Verbose Images. ICLR 2024; Dong et al. Engorgio. ICLR 2025; Liu et al. ReasoningBomb. CCS 2026; Kumar et al. OverThink. arXiv:2502.02542.
- Liu et al. Natural Reflection Backdoor on VLM for AD. arXiv:2505.06413; Biton et al. Adversarial Implications of Variable-Time Inference. AISEC 2023.